

CS202 — Game Project Report

Course: CS202 — Programming Systems | **Group:** 02 | **Date:** 2026-09-02

1. Introduction

This project is a 2D side-scrolling platform game inspired by classic Mario gameplay, implemented in C++20 with SFML 3 for graphics, audio, windowing, and input, and Box2D 3.1.1 for physics and collision handling. Players navigate themed levels, defeat enemies, collect coins and power-ups, activate checkpoints, and reach the flagpole while managing score, lives, and a time limit. The game supports solo play, local cooperative play, and a battle-style minigame for two local players or one player against a heuristic AI opponent. An in-game map editor and JSON-based level and prefab data make it possible to create and test custom layouts without changing the core gameplay code. The project is intended for players who enjoy short, replayable platforming challenges and for demonstrating object-oriented game architecture in an educational desktop application.

1.1 Project Overview

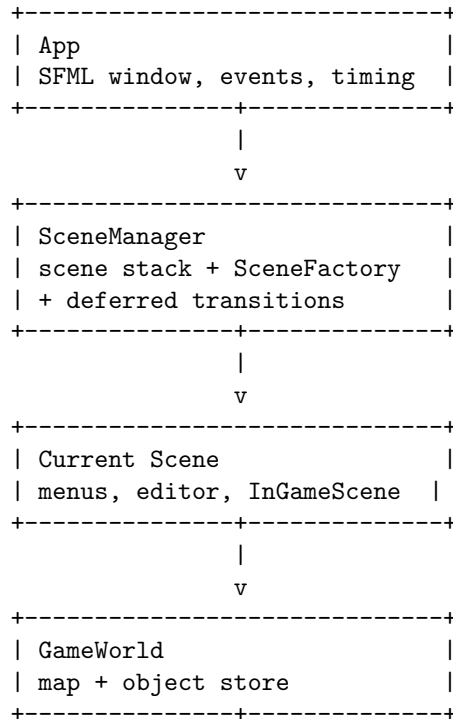
The application begins at a scene-based main menu where players can start or continue a default campaign, select a play mode and character, open settings and leaderboards, or enter the map editor. Default play contains three campaign levels with grassland, underground, and castle-themed content; levels include terrain, slopes, pipes and warps, Goombas, Koopas, Piranha Plants, blocks, coins, power-ups, checkpoints, and flagpole goals. Players may play alone as Mario or Luigi, or play locally with Mario and Luigi together. Minigame mode loads a dedicated arena for two-player competition or a Mario-versus-heuristic-AI match. Campaign sessions persist score, coins, lives, time, checkpoint progress, destroyed tiles, and relevant object state in JSON save files, while completed runs contribute to a persistent leaderboard. The map editor supports placing supported terrain and gameplay objects, validates player spawns, saves a custom JSON map, and allows the result to be playtested in the game.

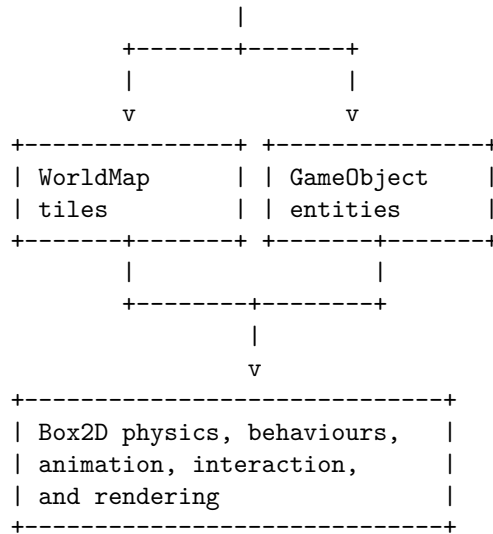
2. System Overview

The system is organized into the following cooperating subsystems:

- **Application and timing:** App owns the SFML window and main loop, polls input events, separates simulation from visual updates, and caps rendering at 60 FPS.

- **Scene management:** `SceneManager` maintains a stack of screens, while `SceneFactory` creates menus, settings, level and character selection, gameplay, save/load, score, leaderboard, and editor scenes.
- **Gameplay world:** `InGameScene` coordinates the active `GameWorld`, camera, HUD, score, pause flow, win conditions, and game-over handling.
- **Map and content loading:** `LevelDataLoader`, `PrefabRegistry`, and `PrefabSpawner` read JSON level data, resolve reusable prefab definitions, build tile terrain, and create gameplay objects from map symbols and placements.
- **Entities and behaviours:** `GameObject` provides the common entity interface for players, enemies, blocks, items, pipes, and projectiles; reusable behaviours, controllers, and player state objects add movement, animation, attacks, damage, power-ups, and AI.
- **Physics and interaction:** `PhysicsWorld` advances Box2D bodies, collision filters, contacts, and sensors; `WorldInteractionSystem` translates those events into gameplay effects such as damage, item collection, enemy defeat, warps, and block destruction.
- **Presentation and persistence:** SFML renders the world, camera view, animations, HUD, effects, and UI, while `ResourceManager` and audio managers provide shared assets; `SaveLoadGame`, `GameSettings`, `ScoreManager`, and `LeaderboardManager` provide JSON-backed session, configuration, scoring, and leaderboard data.





JSON maps/prefabs -> LevelDataLoader / PrefabRegistry -> GameWorld

2.1 Architecture Layers

The top-level control flow is a two-rate loop. `App::run()` polls SFML events, adds the measured frame time to a `FixedStepAccumulator`, consumes zero or more fixed simulation steps, updates visual-only systems using the current frame delta, and renders the frame. During gameplay, the fixed-step path is `App -> SceneManager -> InGameScene -> GameWorld`; the accumulator supplies a simulation delta of 1/60 second, so movement, game rules, and physics are not tied directly to variable render-frame duration. `PhysicsWorld` further advances Box2D using its configured internal substeps, then buffered contact and sensor events are processed by the world interaction system.

The architecture follows a layered ownership model. The application layer owns the window and timing; the scene layer owns screen-specific input, UI, lifecycle, and transitions; the world layer owns the loaded map, entities, object creation, interactions, camera-facing gameplay state, and rendering coordination; and the entity layer models individual actors through the polymorphic `GameObject` base class. Physics bodies and collision data are attached to entities through the physics layer, while animation, audio, and SFML drawing provide the presentation layer. This separation allows menu and editor screens to reuse the same application and scene infrastructure while gameplay scenes reuse world services for campaign levels, custom maps, and minigames.

3. Class Diagram

3.1 Main Architecture

```
App
+-- SceneManager
    +-- Scene
        +-- Menu / Settings / Editor scenes
        +-- InGameScene
            +-- GameWorld
                +-- WorldMap
                +-- WorldObjectStore
                |   +-- GameObject
                |       +-- Player
                |       +-- Enemy
                |       +-- Block
                |       +-- Item
                |       +-- Fireball
                |       +-- KoopaShell
            +-- PhysicsWorld
            +-- Rendering / Interaction
```

3.2 Class Diagram

4. Applied Design Patterns

The project incorporates several classic object-oriented design patterns to structure game flow, manage entity states, decouple subsystems, and promote extensibility.

#	Pattern	Where Applied	Problem Solved
1	State	PlayerState (NormalState, SuperState, FireState), Player::setState(), Scene / SceneManager	Encapsulates form-dependent physics, visuals, damage reactions, and attack capabilities into polymorphic classes, eliminating massive conditional switches.

#	Pattern	Where Applied	Problem Solved
2	Decorator	PlayerStateDecorator (MegaStateDecorator, StarManStateDecorator)	Augments player forms with temporary, expiring effects (Mega size, Star Man invincibility) while transparently preserving and restoring the underlying base state.
3	Strategy	IAttackStrategy (FireballAttackStrategy, NoAttackStrategy), IPlayerController (PlayerController, HeuristicAiController)	Decouples attack mechanics and input control schemes from entity classes, allowing dynamic swapping of attack behaviors and AI/human controls at runtime.
4	Factory Method	GameObjectFactory, SceneFactory	Encapsulates instantiation and configuration of heterogeneous entities (players, enemies, items, blocks, pipes) and scenes from string keys, decoupling loading from concrete classes.
5	Command	ICommand, FunctionalCommand, ButtonMenu	Encapsulates UI button callbacks, menu actions, and debug shortcuts into standalone executable objects, decoupling menu dispatchers from concrete scene operations.
6	Singleton	ResourceManager, GameSettings, Audio::SoundManager, Audio::MusicManager	Provides unified global access and lifecycle management for heavy shared resources (textures, audio streams, fonts, user settings), preventing duplicate GPU/memory allocations.

4.1 Pattern in Detail — State Pattern

Context / Problem:

In a platformer game, the playable character alternates between distinct power-up

forms (Normal/Small, Super, and Fire) with fundamentally different attributes, physics parameters, and gameplay capabilities: - **Normal Mario**: Standard scale (36×80 px), base speed and jump multipliers ($1.0\times$), unable to shoot projectiles, and immediately knocked out / loses a life upon taking damage from enemies. - **Super Mario**: Enlarged scale ($1.5\times$ physical and visual height), boosted jump multiplier ($1.15\times$), ability to break brick blocks from underneath, and degrades to Normal Mario with temporary invincibility frames when damaged rather than dying. - **Fire Mario**: Retains enlarged scale ($1.5\times$), uses a distinct fire costume spritesheet, and gains the capability to cast bouncing fireballs (**Fireball**) on attack input, degrading to Super Mario upon taking damage.

If implemented naively using procedural flags or an enumeration (`enum class Form { Normal, Super, Fire }`), the `Player` class would become congested with complex conditional branches (`switch (form)` or chained `if-else`) across almost every method—such as movement computation, jump physics, collision callbacks, animation assignment, and attack handling. Adding a new form (e.g., Ice Mario) would force modifications across dozens of disjoint switch statements throughout `Player.cpp`, directly violating the Open/Closed Principle (OCP) and Single Responsibility Principle (SRP).

Solution in our code:

We implemented the Gang-of-Four **State Pattern** to encapsulate state-specific logic into dedicated polymorphic classes:

1. **Abstract State Interface (PlayerState)** (`include/Game/Objects/Player/State/PlayerState.h:16`)
Defines the complete polymorphic contract:

- Behavioral multipliers: `getMoveSpeedMultiplier()`, `getJumpSpeedMultiplier()`, `getScaleMultiplier()`.
- Visual configuration: `getStateName()`, `getAnimationSetId()`, `getTextureAlias()`.
- Capabilities & Attack Strategy: `canShootFireballs()`, `createAttackStrategy()` (defaults to `NoAttackStrategy`).
- Lifecycle hooks: `onEnter(Player&)`, `onExit(Player&)`, `update(Player&, float)`.
- Event and transition dispatchers: `handleSuperMushroom(Player&)`, `handleFireFlower(Player&)`, `handleSuperStar(Player&)`, and `handleEnemy(Player&)`.

2. **Concrete State Classes:**

- `NormalState` (`include/Game/Objects/Player/State/NormalState.h`, `src/Game/Objects/Player/State/NormalState.cpp`): Implements baseline $1.0\times$ multipliers, standard character spritesheets, disallows fireball attacks, and routes `handleEnemy()` to `player.knockout()`.
- `SuperState` (`include/Game/Objects/Player/State/SuperState.h`, `src/Game/Objects/Player/State/SuperState.cpp`): Overrides scale to $\{1.5f, 1.5f\}$, jump boost to $1.15\times$, and de-

- `FireState` (`include/Game/Objects/Player/State/FireState.h`, `src/Game/Objects/Player/State/FireState.cpp`): Binds `fire_<character>_spritesheet`, overrides `canShootFireballs()` to return `true`, overrides `createAttackStrategy()` to return `std::make_unique<FireballAttackStrategy>()`, and degrades on enemy contact.

- ```
void Player::setState(std::unique_ptr<PlayerState> newState) {
 if (!newState) return;
 if (_state) _state->onExit(*this);
 _state = std::move(newState);
 _state->onEnter(*this);
 refreshStatePresentation();
}
```

**Consequences:**

- **Adherence to OCP:** New power-up states (e.g. Ice Mario, Tanooki Mario) can be added by creating a new derived class from `PlayerState` without modifying existing state classes or changing `Player`'s core movement loop.
- **Single Responsibility:** Each power-up form encapsulates its own physics multipliers, asset IDs, and damage reduction logic in a single compilation unit.
- **Trade-offs:** Requires small heap allocations when instantiating new states (`std::make_unique`), though state transitions occur infrequently (upon item pickup or damage). Additionally, concrete states hold back-references to the `Player` context during transition calls, creating a controlled bidirectional dependency.

8



*rejected:* In C++, an object's dynamic type is immutable after instantiation. To switch from **NormalPlayer** to **SuperPlayer**, the game would have to destroy the existing **Player** entity and its Box2D physics body, instantiate a new object, and re-bind camera targets, controller inputs, score counters, and scene references. This causes pointer invalidation risks, Box2D fixture reconstruction overhead, and complex state synchronization. - **Alternative 3: Boolean Flags (`bool isSuper`, `bool isFire`):** - *Why rejected:* Causes combinatorial state explosion and invalid states (e.g., both `isSuper` and `isFire` being `true` simultaneously). Bug-prone synchronization is required whenever one flag is enabled and another must be disabled.

---

## 4.2 Pattern in Detail — Decorator Pattern

### Context / Problem:

In addition to permanent form transitions, the game features temporary, time-limited power-ups such as the **Mega Mushroom** (16 seconds of giant scale  $8.0\times$ , environment brick destruction, and invulnerability) and the **Super Star / Star Man** (10 seconds of invincibility, rainbow sparkle visual effects, and lethal enemy contact). These temporary power-ups must: 1. Apply to *any* current player form (Normal, Super, or Fire Mario). 2. Override specific properties (such as scale or invincibility) while transparently delegating other behaviors (like running physics or base animation sets) to the underlying form. 3. Automatically expire after a duration and seamlessly restore the player's exact pre-existing form.

If solved via inheritance subclassing, this would cause a **combinatorial class explosion**: `MegaNormalState`, `MegaSuperState`, `MegaFireState`, `StarNormalState`, `StarSuperState`, `StarFireState`, and compound states (`MegaStarFireState`), resulting in an unmaintainable web of duplicate classes.

### Solution in our code:

We implemented the **Decorator Pattern** via `PlayerStateDecorator` (`include/Game/Objects/Player/State/PlayerStateDecorator.h:10-39`), which inherits from `PlayerState` while wrapping an inner `std::unique_ptr<PlayerState> _wrappedState`:

1. **Decorator Base Class (`PlayerStateDecorator`):** Forwards all base `PlayerState` interface calls (`getMoveSpeedMultiplier()`, `getScaleMultiplier()`, `getTextureAlias()`, `handleEnemy()`) directly to `_wrappedState`.
2. **Concrete Decorators:**
  - `MegaStateDecorator` (`include/Game/Objects/Player/State/MegaStateDecorator.h`, `src/Game/Objects/Player/State/MegaStateDecorator.cpp`): Overrides `getScaleMultiplier()` to return `{8.0f, 8.0f}`, sets `isInvincible()` to `true`, disables fireball attacks by returning

NoAttackStrategy, and maintains `_remainingTime`. During `update()`, it decrements the timer and, upon expiration, signals Player to restore the wrapped state.

- **StarManStateDecorator** (`include/Game/Objects/Player/State/StarManStateDecorator.h`, `src/Game/Objects/Player/State/StarManStateDecorator.cpp`): Overrides `isInvincible()` to `true`, applies speed boosts, and attaches the rainbow sparkle emitter while keeping the base state's scale and animation set.

### 3. Dynamic Application & Restoration (`src/Game/Objects/Player/Player.cpp:351-387`):

When Mario collects a Mega Mushroom, `Player::applyMegaState()` moves the current `_state` into the decorator without destroying it:

```
std::unique_ptr<PlayerState> wrappedState = std::move(_state);
setState(std::make_unique<MegaStateDecorator>(std::move(wrappedState), durationSeconds));
```

When the timer expires, `Player::revertDecoratedState()` extracts the original `SuperState` or `FireState` and reinstating it as `_state`.

**Consequences & Alternatives:** - Eliminates the  $N \times M$  class explosion completely: adding a new base state automatically works with Mega and Star Man without writing any decorator code. - Stacking decorators (e.g. collecting a Star Man while in Mega state) is naturally supported by wrapping decorators around decorators. - *Why not alternatives:* Adding timer floats and boolean flags inside every base state class would pollute permanent states with temporary concerns, violating SRP and bloating `PlayerState`.

---

## 5. Design Reasoning

- The main goal of our architecture was to make the gameplay systems easier to extend while keeping the physics stable and object ownership clear. Instead of putting all the logic into one large class, we separated the program into different systems for scenes, world simulation, physics, rendering, input, animation, and object behaviours. This makes the code easier to follow and lets us modify one system without affecting too many unrelated parts.
- During development, our group tried to follow the SOLID principles when making design decisions. For example, we used abstract interfaces for scenes, separated physics from rendering, and used behaviours for abilities that are not needed by every object. Despite this philosophy, the design changed many times as we added more features, so not every part of the project follows these principles perfectly.

## 5.1 Principles

- Single Responsibility Principle (SRP): We tried to give each major class one main responsibility. **SceneManager** manages scene transitions, **GameWorld** coordinates the gameplay systems, **PhysicsWorld** handles the Box2D simulation, **WorldRenderer** renders the world, and **WorldInteractionSystem** processes contact and sensor events. This prevents all of these responsibilities from being placed inside one large class.
- Open/Closed Principle (OCP): The project can be extended with new scenes, game objects, animations, and level prefabs without rewriting most of the existing code. We use inheritance, factories, registries, and JSON definitions to support this. For example, a level object can be added as a prefab and spawned through the existing map-loading system.
- Liskov Substitution Principle (LSP): Concrete classes can generally be used through their base interfaces. **SceneManager** can manage different classes derived from **Scene**, while the player can use different implementations of **PlayerState**. This means the main program flow does not need separate logic for every concrete implementation.
- Interface Segregation Principle (ISP): We use focused abstractions such as **Behaviour** and **IAttackStrategy** instead of putting every possible operation into one large interface. This means an object only needs to use the functionality that is relevant to it. For example, attack logic is kept separate from movement and animation.
- Dependency Inversion Principle (DIP): High-level systems use abstractions when possible. **SceneManager** works with the abstract **Scene** interface instead of directly handling every concrete scene type. Player attack logic also uses **IAttackStrategy**, which allows the attack implementation to change without rewriting the main player logic.
- Composition over inheritance: Optional abilities such as animation, movement, damage handling, invincibility, visual effects, and shell holding are attached to a **GameObject** as behaviours. This lets each object have only the abilities it needs. It also allows temporary behaviours to be added or removed during gameplay without creating many subclasses for every possible combination.

## 5.2 Key Decisions & Trade-offs

### Behaviour Composition

- Optional abilities are stored in **GameObject::\_behaviours** as a `vector<unique_ptr<Behaviour>>`. Objects can use `addBehaviour<T>()`, `getBehaviour<T>()`, and `removeBehaviour<T>()` to change their abilities at runtime. For example, temporary invincibility can be attached to a

player and removed later without adding another level to the `Player` inheritance hierarchy.

- The trade-off is that behaviours are found using a linear search and `dynamic_cast`. This is simple for the small number of behaviours owned by each object, but it would become less efficient if objects had many behaviours. Removing a behaviour also requires care because other systems may still depend on it.

### Data-Driven Levels and Prefabs

- Levels and reusable objects are defined through JSON. Map symbols refer to entries in `PrefabRegistry`, which resolves them into `SpawnSpec` values. These values can configure the object type, texture, animation, size, collision properties, pipe warps, and contained items. This allows us to create and modify levels without hard-coding every object placement in C++.
- The trade-off is that JSON errors are only discovered when the data is loaded. Invalid prefab names, missing properties, or incorrect values cannot be detected by the C++ compiler. Therefore, the project needs careful data validation and clear loading errors.

### Scene Stack

- `SceneManager` owns a `stack<unique_ptr<Scene>>` and changes screens using push, pop, and replace operations. This gives menus, settings, save screens, level selection, and gameplay a common navigation system. Scene changes are deferred until the current operation finishes, preventing a scene from being destroyed while it is still processing an event.
- The trade-off is that only the top scene is treated as the active screen. Transparent overlays are not fully supported as separate scenes, so the pause menu, victory screen, and game-over screen are managed inside `InGameScene`.

### Player States and Decorators

- Normal, Super, and Fire forms are represented through the `PlayerState` interface. Temporary effects such as Mega and Star Man use `PlayerStateDecorator` to wrap the current state. This preserves the underlying form while adding temporary properties such as a larger scale, invincibility, or modified movement.
- The trade-off is that wrapped states are more difficult to inspect and update than a single state value. Some parts of the program use `dynamic_cast` to check which decorator is active, and care must be taken when adding, removing, or restoring nested states.

### 5.3 Limitations & Future Work

- Although the project applies object-oriented principles, the architecture does not follow them perfectly in every part of the program. As more gameplay features were added, some classes gradually gained more responsibilities than originally intended. This makes these classes more difficult to understand, modify, and test. In future development, these responsibilities could be divided into smaller and more focused classes to improve maintainability and follow the Single Responsibility Principle more closely.
- Some complex systems (Player, GameWorld) also depend directly on concrete classes or globally accessible managers. This creates stronger coupling because a change to one class may require changes in other parts of the program. A better approach would be to introduce more focused abstractions and provide dependencies through constructors. This would make individual systems easier to replace, extend, and test independently.
- The behaviour and player-state systems successfully reduce the need for deep inheritance hierarchies, but they also introduce additional runtime type checking and coordination between objects. Future improvements could provide clearer interfaces for communication between behaviours and reduce the need to identify concrete types. This would help the project follow the Open/Closed and Dependency Inversion Principles more consistently.
- Overall, the current architecture is suitable for the scope of the project and supports its main features. However, further refactoring could improve the consistency of the object-oriented design, reduce coupling between systems, and make the project easier to extend with new gameplay mechanics.

---

## 6. Conclusion

- In conclusion, our group developed a 2D Mario-style platformer using C++20, SFML 3, and Box2D 3.1. Throughout the development process, we learned how object-oriented principles and design patterns can be used to organize a large program containing many connected gameplay systems. We also learned that every design decision involves trade-offs and that an architecture often needs to be reviewed and refactored as the project grows. Although some parts of the design can still be improved, the final architecture supports the required features and provides a reasonable foundation for adding new content in the future.